

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.usbinfoRelease`

1.0.0-SNAPSHOT

September 2026



OSGi®

Copyright © 2000, 2026 Eclipse Foundation

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>".

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
145	USB Information Device Category Specification	7
145.1	Introduction	7
145.2	USBInfoDevice Service	8
145.3	Security	10
145.4	org.osgi.service.usbinfo	11
145.5	References	14

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

145 USB Information Device Category Specification

Version 1.0

145.1 Introduction

The ??? defines a unified and sophisticated way to handle devices attached to a residential gateway or devices found in the home network by using various protocols such as USB, ZigBee, Z-Wave, KNX, UPnP, etc.

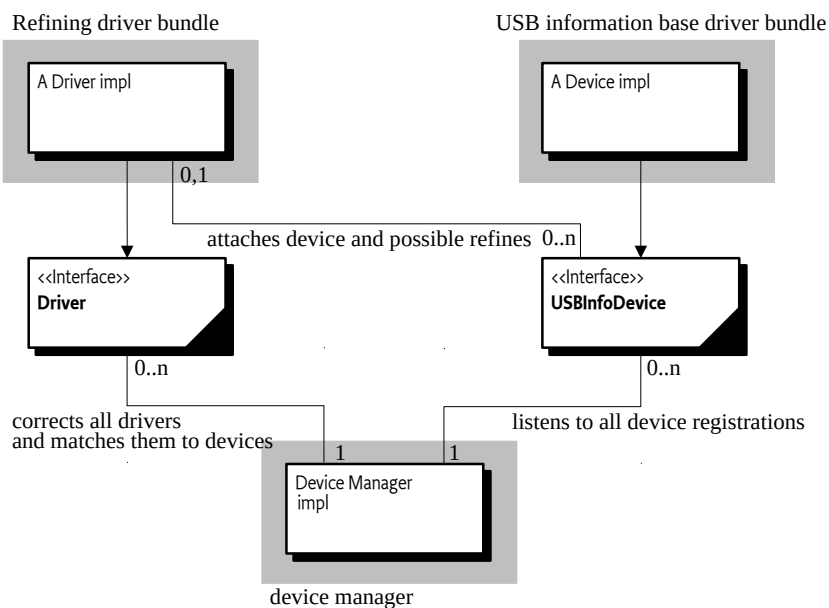
Recently, OSGi is gaining popularity as enabling technology for building embedded systems in the residential market as well as other Internet-of-Things (IoT) domains. Such systems often have USB interfaces and the need of handling USB devices attached to these systems is increasing.

??? defines the concept of device categories. This specification defines a device category for USB devices.

145.1.1 Entities

- *USBInfoDevice* - The representation of a USB device. This service provide information defined by the USB Implementers Forum, Inc.

Figure 145.1 *USB Information Device Service Overview Diagram*



145.2 USBInfoDevice Service

The device services are registered in the OSGi service registry with the [USBInfoDevice](#) interface. The service is registered by a USB information base driver bundle when a USB device is attached. A USB information base driver bundle must implement [USBInfoDevice](#) interface and register the OSGi service under [USBInfoDevice](#) . Refining drivers can find USB devices via USBInfoDevice services and identify the device. The USBInfoDevice service has a set of properties.

USB Specification, see [1] *Universal Serial Bus Specification Revision 1.1* , defines that a USB device has USB interface(s). A USB information base driver bundle must register USBInfoDevice services number of USB interfaces. A USBInfoDevice service has information that contains a USB device information and a USB interface information.

The USB information base driver may need native drivers such as kernel drivers on Linux. This document has a precondition that there are native drivers. It is out of scope how to install native drivers.

145.2.1 Device Access Category

The device access category is called "USBInfo". The category name is defined as a value of [DEVICE_CATEGORY](#) constant. It must be used as a part of the [???](#) service property value on the [USBInfoDevice](#) service. The category defines the following additional service properties for the [USBInfoDevice](#) service.

145.2.2 Service Properties based upon USB Specification

The USB Specification defines a Device Descriptor. USB devices report their attributes using descriptors. The following [USBInfoDevice](#) service properties use information from the USB device descriptor.

Table 145.1 Service properties of USBInfoDevice service from Device Descriptor

The key of service property	Type	Description	Device Descriptor's Field from USB Spec.
usbinfo.bcdUSB	String	OPTIONAL property key. The 4-digit BCD format. Example: "0210"	bcdUSB
usbinfo.bDeviceClass	String	MANDATORY property key. Hexadecimal, 2-digits. Example: "ff"	bDeviceClass
usbinfo.bDeviceSubClass	String	MANDATORY property key. Hexadecimal, 2-digits. Example: "ff"	bDeviceSubClass
usbinfo.bDeviceProtocol	String	MANDATORY property key. Hexadecimal, 2-digits. Example: "ff"	bDeviceProtocol
usbinfo.bMaxPacketSize0	Integer	OPTIONAL property key.	bMaxPacketSize0
usbinfo.idVendor	String	MANDATORY property key. Hexadecimal, 4-digits. Example: "0403"	idVendor

The key of service property	Type	Description	Device Descriptor's Field from USB Spec.
usbinfo.idProduct	String	MANDATORY property key. Hexadecimal, 4-digits. Example: "8372"	idProduct
usbinfo.bcdDevice	String	MANDATORY property key. The 4-digit BCD format. Example: "0200"	bcdDevice
usbinfo.Manufacturer	String	OPTIONAL property key. String value referenced by iManufacturer. The value is not the index value of iManufacturer. Example: "Buffalo Inc."	iManufacturer
usbinfo.Product	String	OPTIONAL property key. String value referenced by iProduct. The value is not the index value of iProduct. Example: "USB2.0 PC Camera"	iProduct
usbinfo.SerialNumber	String	OPTIONAL property key. String value referenced by iSerialNumber. The value is not the index value of iSerialNumber. Example: "57B0002600000001"	iSerialNumber
usbinfo.bNumConfigurations	Integer	OPTIONAL property key.	bNumConfigurations

According to the USB Specification, a device descriptor has some Interface Descriptors.

Refining drivers need each interface descriptor's bInterfaceClass, bInterfaceSubClass and bInterfaceProtocol to identify devices. The following [USBInfoDevice](#) service properties use information from the USB interface descriptor.

Table 145.2

Service properties of USBInfoDevice service from Interface Descriptor

The key of service property	Type	Description	Interface Descriptor's Field from USB Spec.
usbinfo.bInterfaceNumber	Integer	MANDATORY property key.	bInterfaceNumber
usbinfo.bAlternateSetting	Integer	OPTIONAL property key.	bAlternateSetting
usbinfo.bNumEndpoints	Integer	OPTIONAL property key.	bNumEndpoints
usbinfo.bInterfaceClass	String	MANDATORY property key. Hexadecimal, 2-digits. Example: "ff"	bInterfaceClass
usbinfo.bInterfaceSubClass	String	MANDATORY property key. Hexadecimal, 2-digits. Example: "ff"	bInterfaceSubClass
usbinfo.bInterfaceProtocol	String	MANDATORY property key. Hexadecimal, 2-digits. Example: "ff"	bInterfaceProtocol

The key of service property	Type	Description	Interface Descrip- tor's Field from USB Spec.
usbinfo.Interface	String	OPTIONAL property key. String value referenced by iInterface. The value is not the index value of iInterface.	iInterface

145.2.3 Additional Service Properties

Some additional service properties are needed to identify and access a device by refining drivers.

Table 145.3 Additional service properties of USBInfoDevice service

The key of service property	Type	Description
usbinfo.bus	Integer	MANDATORY property key. The value is Integer. Used to identify USB devices with same VID / PID. The value is the ID of the USB bus assigned when connecting the USB device. USB bus ID is integer. The USB bus ID does not change while the USB device remains connected. Example: 3
usbinfo.address	Integer	MANDATORY property key. The value is Integer. Used to identify USB devices with same VID / PID. The value is the ID of the USB address assigned when connecting the USB device. USB address is integer in the range 1-127. The USB address does not change while the USB device remains connected. Example: 2

145.2.4 Match scale

When the driver service is registered by the driver bundle, the Device Manager calls ??? with the argument of the USBInfoDevice service's Service Reference. The driver responds with a match value based on following choices.

- MATCH_VERSION - Constant for the USB device match scale, indicating a match with USB_ID-VENDOR , USB_IDPRODUCT and USB_BCDDEVICE . Value is 50.
- MATCH_MODEL - Constant for the USB device match scale, indicating a match with USB_ID-VENDOR and USB_IDPRODUCT . Value is 40.
- MATCH_PROTOCOL - Constant for the USB device match scale, indicating a match with USB_B-DEVICECLASS , USB_BDEVICESUBCLASS and USB_BDEVICEPROTOCOL , or a match with USB_BINTERFACECLASS , USB_BINTERFACESUBCLASS and USB_BINTERFACEPROTOCOL . Value is 30.
- MATCH_SUBCLASS - Constant for the USB device match scale, indicating a match USB_B-DEVICECLASS and USB_BDEVICESUBCLASS , or a match with USB_BINTERFACECLASS and USB_BINTERFACESUBCLASS . Value is 20.
- MATCH_CLASS - Constant for the USB device match scale, indicating a match with USB_BDE-VICECLASS , or a match with USB_BINTERFACECLASS . Value is 10.

145.3 Security

To acquire USB information device service, The refining bundle require that ServicePermis-sion[USBInfoDevice, GET] is assigned.

USBInfoDevice service should only be implemented by trusted bundles. This bundle requires ServicePermission[USBInfoDevice, REGISTER].

145.4 org.osgi.service.usbinfo

USB Information Device Category Specification Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.usbinfo; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.usbinfo; version="[1.0,1.1)"}
```

145.4.1 Summary

- USBInfoDevice - Represents a USB device.

145.4.2 public abstract USBInfoDevice

Represents a USB device. For each USB device, an object is registered with the framework under the USBInfoDevice interface. A USB information base driver must implement this interface.

The values of the USB property names are defined by the USB Implementers Forum, Inc.

Concurrency Thread-safe

145.4.2.1 public static final String DEVICE_CATEGORY = "USBInfo"

Constant for the value of the service property {@code DEVICE_CATEGORY} used for all USB devices.

A USB information base driver bundle must set this property key.

See Also org.osgi.service.device.Constants.DEVICE_CATEGORY

145.4.2.2 public static final int MATCH_CLASS = 10

Device Access match value indicating a match with {@link #USB_BDEVICECLASS} or a match with {@link #USB_BINTERFACECLASS}.

145.4.2.3 public static final int MATCH_MODEL = 40

Device Access match value indicating a match with {@link #USB_IDVENDOR}, and {@link #USB_IDPRODUCT}.

145.4.2.4 public static final int MATCH_PROTOCOL = 30

Device Access match value indicating a match with {@link #USB_BDEVICECLASS}, {@link #USB_BDEVICESUBCLASS}, and {@link #USB_BDEVICEPROTOCOL} or a match with {@link #USB_BINTERFACECLASS}, {@link #USB_BINTERFACESUBCLASS}, and {@link #USB_BINTERFACEPROTOCOL}.

145.4.2.5 public static final int MATCH_SUBCLASS = 20

Device Access match value indicating a match with {@link #USB_BDEVICECLASS}, and {@link #USB_BDEVICESUBCLASS} or a match with {@link #USB_BINTERFACECLASS}, and {@link #USB_BINTERFACESUBCLASS}.

- 145.4.2.6** **public static final int MATCH_VERSION = 50**
Device Access match value indicating a match with {@link #USB_IDVENDOR}, {@link #USB_ID-PRODUCT}, and {@link #USB_BCDDEVICE}.
- 145.4.2.7** **public static final String USB_ADDRESS = "usbinfo.address"**
Service property to identify USB address.
Used to identify USB devices with same VID / PID. The value is the ID of the USB address assigned when connecting the USB device. USB address is an integer in the range 1-127 and does not change while the USB device remains connected. The value type is Integer.
- 145.4.2.8** **public static final String USB_BALTERNATESETTING = "usbinfo.bAlternateSetting"**
Service property for USB Interface Descriptor field "bAlternateSetting".
The value type is Integer. This service property is optional.
- 145.4.2.9** **public static final String USB_BCDDEVICE = "usbinfo.bcdDevice"**
Service property for USB Device Descriptor field "bcdDevice".
The value type is String; the value is in 4-digit BCD format. For example, "0200".
- 145.4.2.10** **public static final String USB_BCDUSB = "usbinfo.bcdUSB"**
Service property for USB Device Descriptor field "bcdUSB".
The value type is String; the value is in 4-digit BCD format. For example, "0210". This service property is optional.
- 145.4.2.11** **public static final String USB_BDEVICECLASS = "usbinfo.bDeviceClass"**
Service property for USB Device Descriptor field "bDeviceClass".
The value type is String; the value is in 2-digit hexadecimal. For example, "ff".
- 145.4.2.12** **public static final String USB_BDEVICEPROTOCOL = "usbinfo.bDeviceProtocol"**
Service property for USB Device Descriptor field "bDeviceProtocol".
The value type is String; the value is in 2-digit hexadecimal. For example, "ff".
- 145.4.2.13** **public static final String USB_BDEVICESUBCLASS = "usbinfo.bDeviceSubClass"**
Service property for USB Device Descriptor field "bDeviceSubClass".
The value type is String; the value is in 2-digit hexadecimal. For example, "ff".
- 145.4.2.14** **public static final String USB_BINTERFACECLASS = "usbinfo.bInterfaceClass"**
Service property for USB Interface Descriptor field "bInterfaceClass".
The value type is String; the value is in 2-digit hexadecimal. For example, "ff".
- 145.4.2.15** **public static final String USB_BINTERFACENUMBER = "usbinfo.bInterfaceNumber"**
Service property for USB Interface Descriptor field "bInterfaceNumber".
The value type is Integer.
- 145.4.2.16** **public static final String USB_BINTERFACEPROTOCOL = "usbinfo.bInterfaceProtocol"**
Service property for USB Interface Descriptor field "bInterfaceProtocol".
The value type is String; the value is in 2-digit hexadecimal. For example, "ff".

- 145.4.2.17** **public static final String USB_BINTERFACESUBCLASS = "usbinfo.bInterfaceSubClass"**
Service property for USB Interface Descriptor field "bInterfaceSubClass".
The value type is String; the value is in 2-digit hexadecimal. For example, "ff".
- 145.4.2.18** **public static final String USB_BMAXPACKETSIZE = "usbinfo.bMaxPacketSize"**
Service property for USB Device Descriptor field "bMaxPacketSize".
The value type is Integer. This service property is optional.
- 145.4.2.19** **public static final String USB_BNUMCONFIGURATIONS = "usbinfo.bNumConfigurations"**
Service property for USB Device Descriptor field "bNumConfigurations".
The value type is Integer. This service property is optional.
- 145.4.2.20** **public static final String USB_BNUMENDPOINTS = "usbinfo.bNumEndpoints"**
Service property for USB Interface Descriptor field "bNumEndpoints".
The value type is Integer. This service property is optional.
- 145.4.2.21** **public static final String USB_BUS = "usbinfo.bus"**
Service property to identify USB bus.
Used to identify USB devices with same VID / PID. The value is the ID of the USB bus assigned when connecting the USB device. The USB bus ID is an integer and does not change while the USB device remains connected. The value type is Integer.
- 145.4.2.22** **public static final String USB_IDPRODUCT = "usbinfo.idProduct"**
Service property for USB Device Descriptor field "idProduct".
The value type is String; the value is in 4-digit hexadecimal. For example, "8372".
- 145.4.2.23** **public static final String USB_IDVENDOR = "usbinfo.idVendor"**
Service property for USB Device Descriptor field "idVendor".
The value type is String; the value is in 4-digit hexadecimal. For example, "0403".
- 145.4.2.24** **public static final String USB_INTERFACE = "usbinfo.Interface"**
Service property for name referenced by USB Interface Descriptor field "iInterface".
The value type is String. This service property is optional.
- 145.4.2.25** **public static final String USB_MANUFACTURER = "usbinfo.Manufacturer"**
Service property for name referenced by USB Device Descriptor field "iManufacturer".
The value type is String. For example, "Buffalo Inc.". This service property is optional.
- 145.4.2.26** **public static final String USB_PRODUCT = "usbinfo.Product"**
Service property for name referenced by USB Device Descriptor field "iProduct".
The value type is String. For example, "USB2.0 PC Camera". This service property is optional.
- 145.4.2.27** **public static final String USB_SERIALNUMBER = "usbinfo.SerialNumber"**
Service property for name referenced by USB Device Descriptor field "iSerialNumber".
The value type is String. For example, "57B0002600000001". This service property is optional.

145.5

References

- [1] *Universal Serial Bus Specification Revision 1.1*
September 23, 1998.

DRAFT

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT